

Lab Work for Week 8: Django Class-Based Views (CBVs)

Objective

By the end of this lab, students will:

1. Understand the purpose and advantages of Class-Based Views (CBVs).
 2. Implement commonly used CBVs (ListView, DetailView, CreateView) in their project.
 3. Learn to customize CBVs for specific use cases.
-

Step-by-Step Lab Tasks

1. Convert Course List and Detail Views to CBVs

1. **Task:** Update views.py in the courses app to use CBVs for listing courses and viewing course details.
 - Replace the function-based views with CBVs:

```
from django.views.generic import ListView, DetailView
```

```
from .models import Course
```

```
class CourseListView(ListView):
```

```
    model = Course
```

```
    template_name = 'courses/course_list.html'
```

```
    context_object_name = 'courses'
```

```
class CourseDetailView(DetailView):
```

```
    model = Course
```

```
    template_name = 'courses/course_detail.html'
```

```
    context_object_name = 'course'
```

2. **Why This Matters:** CBVs reduce boilerplate code and provide built-in methods for common use cases.

2. Update URLs for CBVs

1. **Task:** Update the `urls.py` file in the `courses` app to reference the CBVs.

- Modify `urls.py` as follows:

```
from django.urls import path
```

```
from .views import CourseListView, CourseDetailView
```

```
urlpatterns = [  
    path("", CourseListView.as_view(), name='course_list'),  
    path('<int:pk>/', CourseDetailView.as_view(), name='course_detail'),  
]
```

2. **Why This Matters:** The `.as_view()` method connects CBVs to URL patterns.
-

3. Implement a CreateView for Adding Courses

1. **Task:** Add a `CreateView` in `views.py` to allow instructors to add new courses.

- Add the following code:

```
from django.views.generic.edit import CreateView
```

```
from django.urls import reverse_lazy
```

```
class CourseCreateView(CreateView):  
    model = Course  
    fields = ['title', 'description', 'duration', 'thumbnail']  
    template_name = 'courses/course_form.html'  
    success_url = reverse_lazy('course_list')
```

2. **Why This Matters:** `CreateView` simplifies the process of creating new records and handling forms.
-

4. Add URLs for the CreateView

1. **Task:** Add a URL pattern for the `CourseCreateView` in `urls.py`.

- Update `urls.py`:

```
from .views import CourseCreateView
```

```
urlpatterns = [  
    path("", CourseListView.as_view(), name='course_list'),  
    path('<int:pk>/', CourseDetailView.as_view(), name='course_detail'),  
    path('create/', CourseCreateView.as_view(), name='course_create'),  
]
```

2. **Why This Matters:** This allows instructors to access the course creation page.
-

5. Create Templates for CBVs

1. **Task:** Update course_list.html to display the course list.
 - o Modify the template:

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Courses</title>  
</head>  
  
<body>  
    <h1>Available Courses</h1>  
    <ul>  
        {% for course in courses %}  
            <li>  
                <a href="{{ course.pk }}">{{ course.title }}</a>  
            </li>  
        {% endfor %}  
    </ul>  
    <a href="create/">Add a New Course</a>  
</body>  
</html>
```

2. **Task:** Create `course_form.html` for the `CreateView`.

- Add the following code:

```
<!DOCTYPE html>

<html>

<head>

  <title>Add Course</title>

</head>

<body>

  <h1>Add a New Course</h1>

  <form method="post" enctype="multipart/form-data">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Save</button>

  </form>

  <a href="/">Back to Course List</a>

</body>

</html>
```

3. **Why This Matters:** Templates are tailored to work seamlessly with CBVs.

6. Test CBVs

1. **Task:** Start the development server.

```
python manage.py runserver
```

2. **Task:** Perform the following tests:

- Visit the course list page (`/`) and ensure all courses are displayed.
 - Click on a course to view its details (`/<course_id>/`).
 - Navigate to the course creation page (`/create/`) and add a new course.
 - Verify that the new course appears in the course list.
-

Lab Deliverables

1. Code for:

- Updated views.py using ListView, DetailView, and CreateView.
- Updated urls.py to include CBVs.
- Templates: course_list.html, course_detail.html, and course_form.html.

2. Screenshots of:

- Course list page.
- Course detail page.
- Course creation form and success.

Additional Questions for Students

1. What are the advantages of CBVs compared to function-based views?
2. How does reverse_lazy work in CreateView?
3. How can you customize a CBV's behavior using class methods?